

Group:

Allen Lu (yl10118)

Ryan Wong (rlw9891)

Instructions to run the program:

```
python3 h1.py <input.txt> <output.txt>
```

```
python3 h2.py <input.txt> <output.txt>
```

Output files:output1h1.txt

7 1 6

8 3 5

2 0 4

8 7 6

1 0 5

2 3 4

5

12

U U L D R

5 5 5 5 5

output1h2.txt

7 1 6

8 3 5

2 0 4

8 7 6

1 0 5

2 3 4

5

12

U U L D R

5 5 5 5 5

output2h1.txt

2 6 0

1 3 4

7 5 8

1 2 3
4 5 6
7 8 0

10
27
L D R U L L D R D R
10 10 10 10 10 10 10 10 10 10

output2h2.txt

2 6 0
1 3 4
7 5 8

1 2 3
4 5 6
7 8 0

10
24
L D R U L L D R D R
10 10 10 10 10 10 10 10 10 10

output3h1.txt

8 7 3
0 4 5
6 2 1

1 2 3
4 5 6
7 8 0

23
1141
U R D D R U L D L U R D R D L L U U R D R D
17 17 17 19 19 19 21 23 23 23 23 23 23 23 23 23 23 23 23 23

output3h2.txt

8 7 3
0 4 5

6 2 1

1 2 3

4 5 6

7 8 0

23

756

URDDRULDLUURDRDLLUURDRD

17 17 17 19 19 19 21 23 23 23 23 23 23 23 23 23 23 23 23 23

Source Code:

h1.py

```
# h1.py
import heapq
import copy
import sys

def read_input(filename):
    """Returns the start and goal lists"""
    # Read in the input
    with open(filename, "r") as f:
        lines = [line.strip() for line in f.readlines() if line.strip() != ""]

    # Initialize the start and goal lists
    start = []
    goal = []
    for i in range(3):
        row = []
        for num in lines[i].split():
            row.append(int(num))
        start.append(row)
    for i in range(3, 6):
        row = []
        for num in lines[i].split():
            row.append(int(num))
        goal.append(row)
    return start, goal
```

```

def write_output(filename, start, goal, depth, total_nodes, moves, f_values):
    """Writes the output file"""
    with open(filename, "w") as f:
        for row in start:
            f.write(" ".join([str(num) for num in row]) + "\n")
        f.write("\n")
        for row in goal:
            f.write(" ".join([str(num) for num in row]) + "\n")
        f.write("\n")
        f.write(str(depth) + "\n")
        f.write(str(total_nodes) + "\n")
        f.write(" ".join(moves) + "\n") # Write all of the moves, separated by
a space
        f.write(" ".join([str(f_value) for f_value in f_values])) # Write each
f value, separated by a space

def flatten(board):
    """Flatten the 2D board into a tuple so it can be hashed"""
    return tuple(num for row in board for num in row)

def find_tile(board, val):
    """Return the row and column of the number in the board"""
    for row in range(3):
        for col in range(3):
            if board[row][col] == val:
                return row, col

def manhattan(board, goal):
    """Return total Manhattan distance heuristic"""
    dist = 0
    for row in range(3):
        for col in range(3):
            val = board[row][col]
            if val == 0:

```

```

        continue

        g_row, g_col = find_tile(goal, val) # Find the coordinates of the
number in the goal state

        dist += abs(row-g_row) + abs(col-g_col)

    return dist

class Node:

    def __init__(self, board, g, h, parent=None, move=None):
        self.board = board
        self.g = g
        self.h = h
        self.f = g + h
        self.parent = parent
        self.move = move

    def __lt__(self, other):
        """Returns if this node has a smaller f value than the other node"""
        return self.f < other.f

def get_neighbors(board):
    """Returns possible moves (U, D, L, R) and their boards"""
    moves = []
    row, col = find_tile(board, 0)
    directions = {
        'U': (row - 1, col),
        'D': (row + 1, col),
        'L': (row, col - 1),
        'R': (row, col + 1)
    }
    for move, (new_row, new_col) in directions.items():
        if 0 <= new_row < 3 and 0 <= new_col < 3: # If in the board still
            new_board = copy.deepcopy(board)

            # Swap places
            new_board[row][col], new_board[new_row][new_col] =
new_board[new_row][new_col], new_board[row][col]

```

```

        moves.append((move, new_board))
    return moves

def reconstruct_path(node):
    """Returns moves and f values for the whole path"""
    moves = []
    f_values = []

    # Add each move and f value to list, from goal to start
    while node:
        f_values.append(node.f)
        if node.move:
            moves.append(node.move)
        node = node.parent

    moves.reverse()
    f_values.reverse()
    return moves, f_values

def a_star(start, goal):
    """Returns the depth, total nodes, moves, and f values"""
    start_h = manhattan(start, goal)
    start_node = Node(start, 0, start_h)
    open_list = []
    closed = set()
    heapq.heappush(open_list, start_node)
    total_nodes = 1

    # Go through unexpanded nodes, in order of f value
    while open_list:
        current = heapq.heappop(open_list)
        if current.board == goal:
            moves, f_values = reconstruct_path(current)
            return len(moves), total_nodes, moves, f_values
        closed.add(flatten(current.board))

```

```

        for move, new_board in get_neighbors(current.board):
            new_flat = flatten(new_board)
            if new_flat in closed:
                continue
            g = current.g + 1
            h = manhattan(new_board, goal)
            new_node = Node(new_board, g, h, current, move)
            heapq.heappush(open_list, new_node)
            total_nodes += 1

    return None # Solution not found

def main():

    # Ensure the correct amount of parameters
    if len(sys.argv) != 3:
        print("Use this format: python3 h1.py <input.txt> <output.txt>")
        return

    input_file = sys.argv[1]
    output_file = sys.argv[2]
    start, goal = read_input(input_file)
    depth, total_nodes, moves, f_values = a_star(start, goal)
    write_output(output_file, start, goal, depth, total_nodes, moves, f_values)

if __name__ == "__main__":
    main()

```

h2.py

```
# h2.py
from h1 import * # use some functions that were defined in
import sys
import heapq

def linear_conflict(board, goal):
    """count the number of linear conflicts in rows and columns"""
    conflicts = 0 # initial counter

    # CHECK ROWS
    for row in range(3):
        for col in range(3):
            val = board[row][col]
            if val == 0: # if empty tile
                continue
            goal_row, goal_col = find_tile(goal, val)
            if row == goal_row:
                for col2 in range(col + 1, 3):
                    val2 = board[row][col2]
                    if val2 == 0:
                        continue
                    goal_row2, goal_col2 = find_tile(goal, val2)

                    if goal_row2 == row and goal_col2 < goal_col:
                        conflicts += 1

    # CHECK COLUMNS
    for col in range(3):
        for row in range(3):
            val = board[row][col]
            if val == 0:
                continue

            goal_row, goal_col = find_tile(goal, val)
            if col == goal_col:
                for row2 in range(row + 1, 3):
```

```

        val2 = board[row2][col]
        if val2 == 0:
            continue
        goal_row2, goal_col2 = find_tile(goal, val2)

        if goal_col2 == col and goal_row2 < goal_row:
            conflicts += 1

    return conflicts

def h2_heuristic(board, goal):
    """h2 = h1 + 2 * number of linear conflicts"""
    return manhattan(board, goal) + 2 * linear_conflict(board, goal)

def a_star(start, goal):
    """Returns the depth, total nodes, moves, and f values"""
    start_h = h2_heuristic(start, goal) # uses h2 heuristic here

    start_node = Node(start, 0, start_h)
    open_list = []
    closed = set()
    heapq.heappush(open_list, start_node)
    total_nodes = 1

    # Go through unexpanded nodes, in order of f value
    while open_list:
        current = heapq.heappop(open_list)
        if current.board == goal:
            moves, f_values = reconstruct_path(current)
            return len(moves), total_nodes, moves, f_values
        closed.add(flatten(current.board))
        for move, new_board in get_neighbors(current.board):
            new_flat = flatten(new_board)
            if new_flat in closed:

```

```

        continue

    g = current.g + 1
    h = h2_heuristic(new_board, goal) # uses h2 heuristic here
    new_node = Node(new_board, g, h, current, move)
    heapq.heappush(open_list, new_node)
    total_nodes += 1

return None # Solution not found

def main():
    # Ensure the correct amount of parameters
    if len(sys.argv) != 3: # if not 3 arguments print out the error message
        print("Use this format: python3 h2.py <input.txt> <output.txt>")
        return

    input_file = sys.argv[1]
    output_file = sys.argv[2]
    start, goal = read_input(input_file)
    depth, total_nodes, moves, f_values = a_star(start, goal)
    write_output(output_file, start, goal, depth, total_nodes, moves, f_values)

if __name__ == "__main__":
    main() # main

```